

Spring Integration on SAP NetWeaver



SAP Platform Ecosystem



Copyright

© Copyright 2005 SAP AG. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP AG. The information contained herein may be changed without prior notice.

Some software products marketed by SAP AG and its distributors contain proprietary software components of other software vendors.

Microsoft, Windows, Outlook, and PowerPoint are registered trademarks of Microsoft Corporation.

IBM, DB2, DB2 Universal Database, OS/2, Parallel Sysplex, MVS/ESA, AIX, S/390, AS/400, OS/390, OS/400, iSeries, pSeries, xSeries, zSeries, z/OS, AFP, Intelligent Miner, WebSphere, Netfinity, Tivoli, and Informix are trademarks or registered trademarks of IBM Corporation in the United States and/or other countries.

Oracle is a registered trademark of Oracle Corporation.

UNIX, X/Open, OSF/1, and Motif are registered trademarks of the Open Group.

Citrix, ICA, Program Neighborhood, MetaFrame, WinFrame, VideoFrame, and MultiWin are trademarks or registered trademarks of Citrix Systems, Inc.

HTML, XML, XHTML and W3C are trademarks or registered trademarks of W3C®, World Wide Web Consortium, Massachusetts Institute of Technology.

Java is a registered trademark of Sun Microsystems, Inc.

JavaScript is a registered trademark of Sun Microsystems, Inc., used under license for technology invented and implemented by Netscape.

MaxDB is a trademark of MySQL AB, Sweden.

SAP, R/3, mySAP, mySAP.com, xApps, xApp, SAP NetWeaver, and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP AG in Germany and in several other countries all over the world. All other product and service names mentioned are the trademarks of their respective companies. Data contained in this document serves informational purposes only. National product specifications may vary.

These materials are subject to change without notice. These materials are provided by SAP AG and its affiliated companies ("SAP Group") for informational purposes only, without representation or warranty of any kind, and SAP Group shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP Group products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

Icons in Body Text

Icon	Meaning
	Caution
	Example
	Note
	Recommendation
	Syntax

Additional icons are used in SAP Library documentation to help you identify different types of information at a glance. For more information, see *Help on Help → General Information Classes and Information Classes for Business Information Warehouse* on the first page of any version of *SAP Library*.

Typographic Conventions

Type Style	Description
<i>Example text</i>	Words or characters quoted from the screen. These include field names, screen titles, pushbuttons labels, menu names, menu paths, and menu options. Cross-references to other documentation.
Example text	Emphasized words or phrases in body text, graphic titles, and table titles.
EXAMPLE TEXT	Technical names of system objects. These include report names, program names, transaction codes, table names, and key concepts of a programming language when they are surrounded by body text, for example, SELECT and INCLUDE.
Example text	Output on the screen. This includes file and directory names and their paths, messages, names of variables and parameters, source text, and names of installation, upgrade and database tools.
Example text	Exact user entry. These are words or characters that you enter in the system exactly as they appear in the documentation.
<Example text>	Variable user entry. Angle brackets indicate that you replace these words and characters with appropriate entries to make entries in the system.
EXAMPLE TEXT	Keys on the keyboard, for example, F2 or ENTER.

Table of Contents

Introduction	5
Objectives.....	5
Prerequisites	5
Introducing the sample application.....	6
Importing the sample application	9
Procedure.....	9
Result.....	9
Creating the EJB Module Project	10
Procedure.....	10
Result.....	10
Referencing Spring libraries in EJB Module Project.....	11
Procedure.....	11
Result.....	11
Creating the EJB Interfaces	12
Procedure.....	12
Result.....	12
Implementing Spring EJB Support	13
Procedure.....	13
Result.....	15
Configuring a shared application context and running the application.....	16
Procedure.....	16
Result.....	18
About the Author	19

Introduction

The Spring Framework offers lots of useful features to help developers with everyday development issues and Spring based applications become more and more widespread. Therefore, in the course of system consolidation and the consequent J2EE migration task on SAP Web AS 6.40, Spring related issues frequently emerge.

Although *J2EE without EJB* is one of Springs main concepts, when migrating a Spring application to SAP Web AS it might be desirable to deploy the existing business logic as an EJB e.g. in order to share its groundbreaking functionality with other J2EE applications.

Springs EJB integration framework therefore provides abstract base classes to create Stateless Session Beans, Stateful Session Beans and Message-Driven Beans. These convenient classes don't relieve you of implementing your EJB interfaces. They rather help you to factor your business logic out of the EJB bean class into a POJO in order to support test driven development.

The approach of implementing business logic in POJOs and using the EJB as a facade is not new to J2EE Developers, but there are advantages in using Springs EJB Support. One of those is that you don't have to implement additional logic into your Bean classes to locate and create the business objects. Thus you can switch your business object implementations easily by editing the bean definitions in Spring's context definition instead of the EJBs source code.

This How-To Guide addresses two major challenges involved here:

- EJB classes extending Springs abstract Base classes for EJB support require their own context configuration inside the EJB Module.
- Your Spring application context has to be reconfigured to ensure your web application still can access the beans located in your EJB Module.

So what we are looking for is a "shared" application context between the Web and EJB Module.

Objectives

In this document we will:

- Deploy a Spring bean (POJO implemented business object) as a Stateless Session Bean by using Springs EJB Integration Framework
- Set up a shared application context so that the web application can still access the business bean implementation via the internal Spring context



Note that this Guide is not intended to introduce the Spring framework and its concepts itself. See the following links for more information on Spring framework:

<http://www.springframework.org>

[Introducing the Spring Framework](#) (TheServerSide.com)

Prerequisites

- ☐ SAP Web AS – Java 6.40 SP11
- ☐ NetWeaver Developer Studio SP11
- ☐ Basic understanding of Springs context and bean factory concepts
- ☐ Download TutIntegratingSpring.zip (Sample application as NWDS EAR and Web Module Projects)

Introducing the sample application

Our sample application is a most simple web application formally running on a Jakarta Tomcat 5.0.28. The application has been migrated to the SAP Web AS 6.40 (Migration is covered in How-To Guide “**Migrating a Spring Based Web Application to SAP NetWeaver**”).

In this guide we will extend this application step by step. If you want to have a quick look on how it's done without doing the whole step by step guide, you can download `TutSpringEJBsupport.zip`. The archive contains the result application of this guide. Simply import the NWDS Projects into your NetWeaver Developer Studio. Information on importing existing projects can be found on SAP Help under [Importing Developer Studio J2EE Projects](#).

First of all Let's have a quick look at our sample application:

The applications business logic is implemented via a single “hello world” POJO object *Hello.java* with a business method *doSomeBusiness()* simply returning a *String*. The web front end is implemented using the Spring MVC framework where *AppController.java* acts, guess what, as the Spring MVC Controller.

Hello.java

```
package com.sap.spring.bl;

public class Hello {
    public String doSomeBusiness(){
        return "Doing some business...";
    }
}
```

The bean configuration for *Hello.java* is done in *springapp-servlet.xml* and it is directly injected into the *appController* bean. Note that no explicit context configuration via any *applicationContext.xml* file is necessary.

springapp-servlet.xml

```
<beans>
..
<bean id="appController"
class="com.sap.spring.web.AppController">
    <property name="hello">
        <ref bean="helloBean"/>
    </property>
</bean>
<bean id="helloBean" class="com.sap.spring.bl.Hello" />
    <bean id="messageSource"
        class="org.springframework.context.support.ResourceBundleMessageSource">
        <property name="basename">
            <value>messages</value>
        </property>
    </bean>
    <bean id="urlMapping"
        class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
        <property name="mappings">
            <props>
                <prop key="/hello.htm">appController</prop>
            </props>
        </property>
    </bean>
</beans>
```

```
</property>
</bean>
</beans>
```

In *AppController.java* the business method *doSomeBusiness()* of *Hello.java* is called and the return value is propagated to a *ModelAndView* object which then is used by the *DispatcherServlet* which is defined in *web.xml* and is loaded on startup

AppController.java

```
package com.sap.spring.web;

import..

import com.sap.spring.bl.Hello;

public class AppController implements Controller {

    private Hello hello;

    public ModelAndView handleRequest(HttpServletRequest request,
        HttpServletResponse response) throws ServletException, IOException {

        Map model = new HashMap();
        model.put("helloString", getHello().doSomeBusiness());
        return new ModelAndView("hello", "model", model);
    }

    public void setHello(Hello hello) {
        this.hello = hello;
    }

    public Hello getHello() {
        return hello;
    }
}
```

In *hello.jsp* the String finally is extracted from the model and printed to the screen (see Figure 1):

```
hello.jsp
<%@ taglib prefix="c" uri="http://java.sun.com/jstl/core" %>
<%@ taglib prefix="fmt" uri="http://java.sun.com/jstl/fmt" %>

<html>
<head><title><fmt:message key="title"/> </title></head>
<body>
<h3><fmt:message key="heading"/></h3>

Message from HelloBean in application context:<br>
  <b><c:out value="${model.helloString}"/></b>

</body>
</html>
```

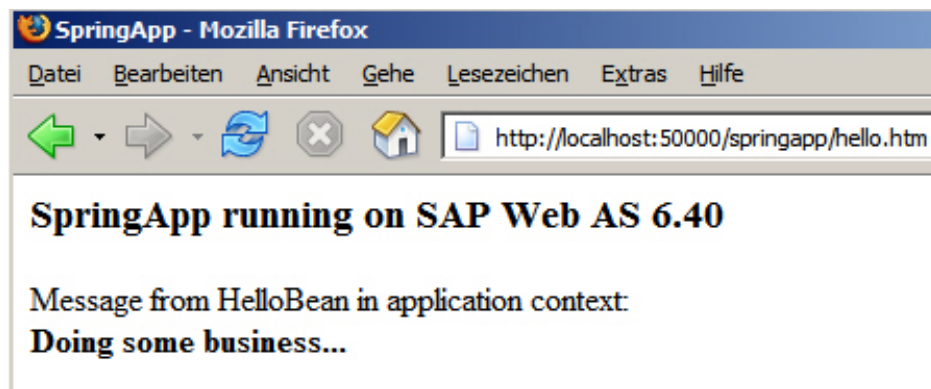


Figure 1 - Sample application running on SAP Web AS 6.40

Importing the sample application

First of all we're going to import the sample application's EAR and Web Module projects into NetWeaver Developer Studio.

Procedure

1. Download `TutIntegratingSpring.zip` archive and extract the folders *TutIntegratingSpring* and *TutIntegratingSpringWeb* anywhere on your system. These are the NetWeaver Developer Studio *EAR*- and the *Web Module Project* of the sample application.
2. Download `TutSpringEJBSupport.zip`. This archive contains the result application of this guide.
3. From *TutSpringEJBSupport.zip* extract *TutSpringEJBSupportLibs* project anywhere on your system. This project is a simple NWDS Java Project which contains all spring libraries used later on in this guide.
4. Import all projects into your NetWeaver Developer Studio Workspace. Information on importing existing projects can be found on SAP Help under [Importing Developer Studio J2EE Projects](#).
5. Remove the spring libraries *spring-aop*, *spring-beans.jar*, *spring-context.jar* and *spring-core.jar* from Web Module Project's *WEB-INF/lib* directory
6. Also remove those libs from the Web Module Project's java build path.
7. Add all Spring libraries from *TutSpringEJBSupportLibs* to the EAR Project's archive build info. This is done in the project's properties (see Figure 2)

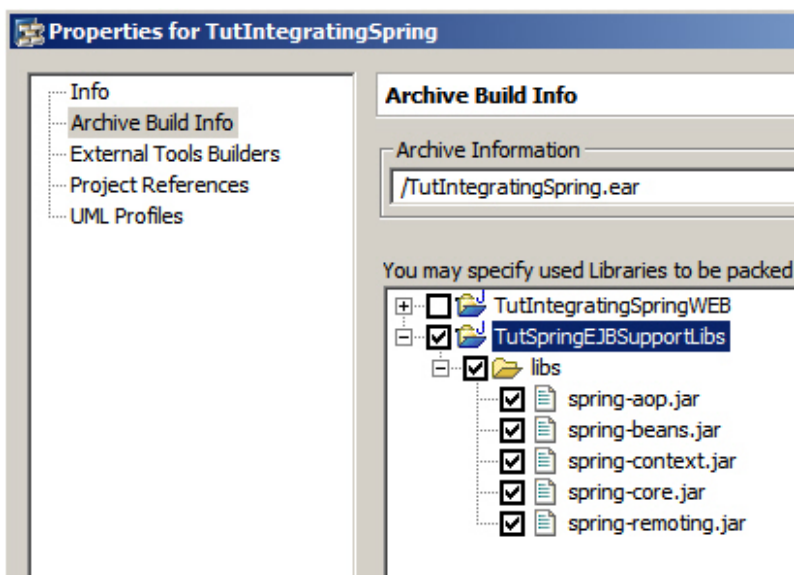


Figure 2 - Adding spring libs to EAR archive

Result

All projects are visible in the NetWeaver Developer Studio *J2EE Explorer* view.

Creating the EJB Module Project

Now we create an *EJB Module Project* which will contain the business logic implementation classes (*Hello.java*) and the EJB interfaces. For detailed information on creating EJB Modules see [Creating EJB Module Projects](#) on SAP Help.

Procedure

1. Create a new EJB Module Project named "TutIntegratingSpringEJB". As a result the new project appears in the *J2EE Explorer*.
2. Create a new package *com.sap.spring.bl* in the projects *ejbModule* folder.
3. Move *Hello.java* from the Web Module Project into the newly created package in the EJB Module Project.
4. Delete the package *com.sap.spring.bl* in the web Web Module Project.
5. Eliminate the indicated build path errors by adding a project reference to *TutIntegratingSpringEJB* in the Web Module Projects java build path (see Figure 3).

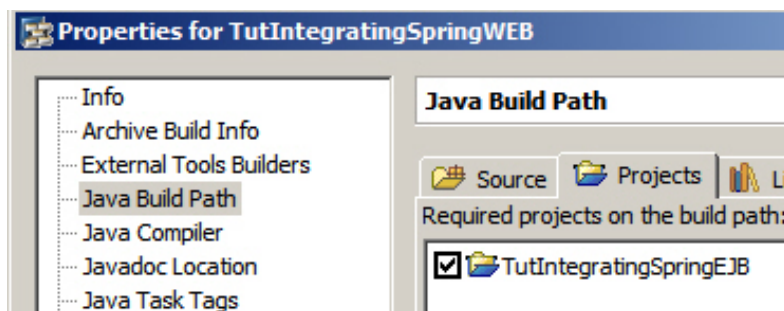


Figure 3 - Project reference in java build path

6. Add the *EJB Module Project* to the Enterprise Application Project by right clicking the *EJB Module Project* and then clicking *Add to EAR Project* in the context menu.
7. Select *TutIntegratingSpring* EAR Project and press *OK*.
8. Save the project.

Result

The whole business logic is now located in our EJB Module Project (see Figure 4). The EAR Project *TutIntegratingSpring* can still be deployed and run. But yet there is no EJB implemented.

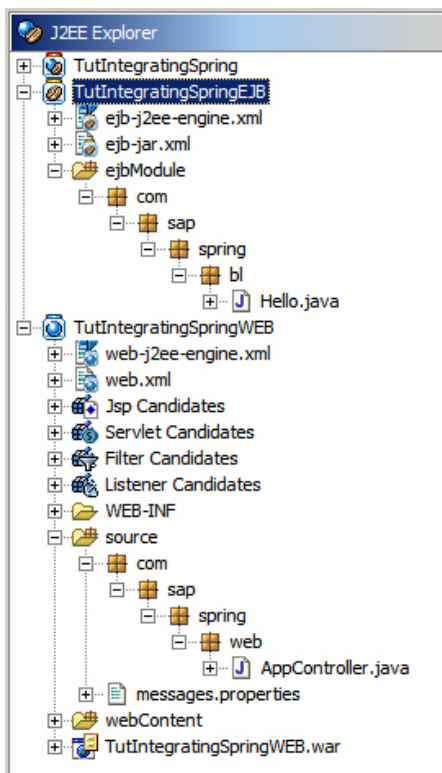


Figure 4 - Business logic located in EJB Module

Referencing Spring libraries in EJB Module Project

In Order to use Springs EJB Support we have to make sure that all necessary libraries are available in our EJB projects build path.

Procedure

1. Reference all spring libraries located in *TutSpringEJBSupport* in the EJB Module Project's properties (see Figure 5).

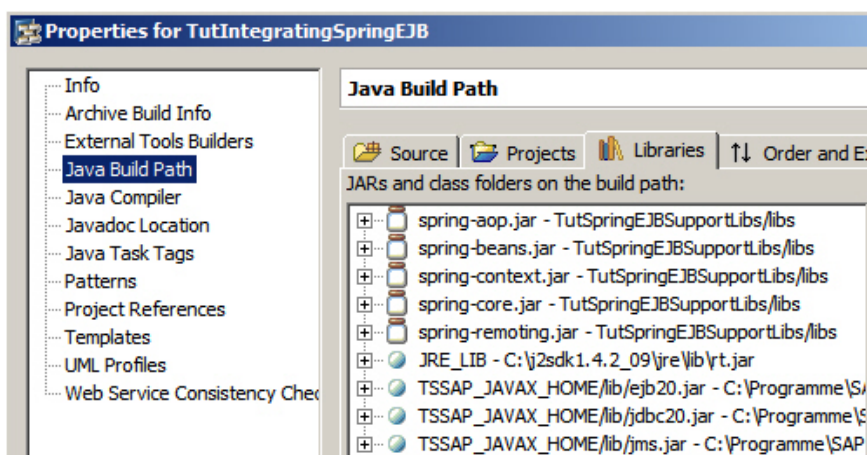


Figure 5 - Spring libraries in java build path

Result

The EJB Modules java build path contains the above libraries.

Creating the EJB Interfaces

Now that all required libraries are available we're going to create all EJB via the NetWeaver Developer Studio's EJB Creation Wizard.

Procedure

1. Create a simple interface *IHello.java* in package *com.sap.spring.bl*. The interface implements the method signature **public String doSomeBusiness()**.
2. Create a new package *com.sap.spring.ejb* in the EJB Module's *ejbModule* folder.
3. Via the EJB creation wizard create a new *Stateless Session Bean* in package *com.sap.spring.ejb* with name "Hello" with a business method *doSomeBusiness()* that returns a *String*. For now only create the local bean interfaces.

 More information on creating Stateless Session Beans can be found under [Creating Stateless Session Beans on SAP Help](#).

Result

In your EJB Module Project the package *com.sap.spring.bl* contains the two classes *Hello.java* and *IHello.java*. The package *com.sap.spring.ejb* contains the automatically generated classes *HelloBean.java*, *HelloLocal.java* and *HelloLocalHome.java* (see Figure 6).

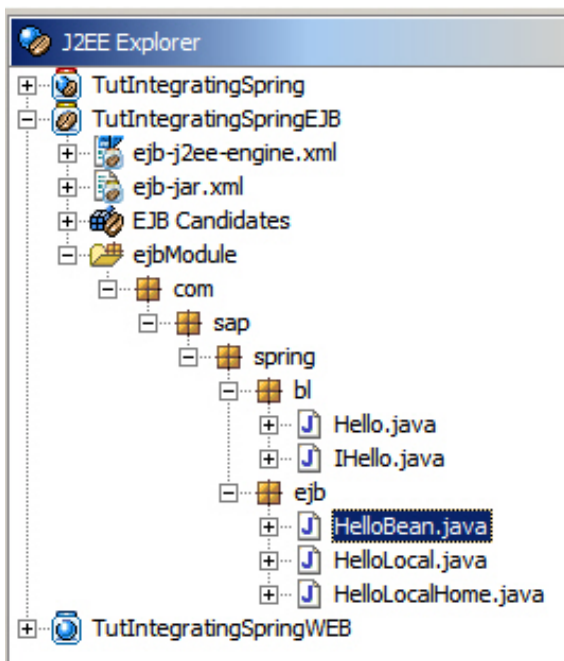


Figure 6 - Generated EJB classes

Implementing Spring EJB Support

By now we have created all EJB interfaces the way we're used to do it. In order to use Spring's EJB Support we have to edit the generated sources. Figure 7 shows what we're up to:

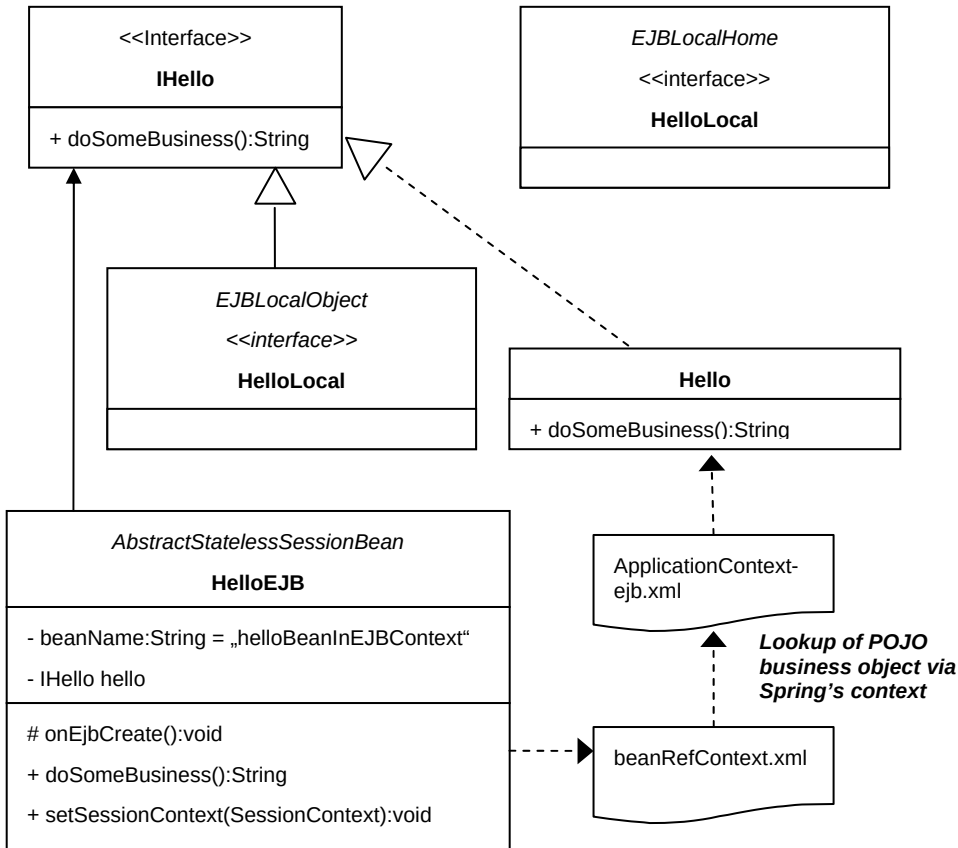


Figure 7 - Class diagram Spring EJB implementation

Procedure

1. Add a new source folder named *conf* to the EJB Module Project (see "Creating a new source folder" in NetWeaver Developer Studio Help). This is where the class loader will find the spring configuration files.
2. Create the files *applicationContext-ejb.xml* and *beanRefContext.xml* in the newly created *conf* folder.

beanRefContext.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN//EN"
"http://www.springframework.org/dtd/spring-beans.dtd">
<beans>
  <bean id="applicationContext-main"
class="org.springframework.context.support.ClassPathXmlApplicationContext">
    <constructor-arg>
      <list><value>applicationContext-ejb.xml</value></list>
    </constructor-arg>
  </bean>
</beans>

```

applicationContext-ejb.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN//EN"
"http://www.springframework.org/dtd/spring-beans.dtd">
<beans>
  <bean id="helloBeanInEJBContext" class="com.sap.spring.bl.Hello"/>
</beans>

```

3. Edit *HelloLocal.java* so that it also extends our own interface *IHello*. Thus, *HelloLocal.java* also doesn't have to implement the business methods signature anymore. By doing this, we assure the correct implementation if the method signature changes in future.

HelloLocal.java

```
package com.sap.spring.ejb;
import javax.ejb.EJBLocalObject;
import com.sap.spring.bl.IHello;

public interface HelloLocal extends EJBLocalObject, IHello {}
```

4. Edit *HelloBean.java* to extend Spring's *AbstractStatelessSessionBean* and implement the overridden methods in the new super class. All regular callback methods such as *ejbRemove*, *ejbCreate* and so on are implemented by Spring classes so we don't need to implement them anymore:

HelloBean.java

```
package com.sap.spring.ejb;

import ..

public class HelloBean extends AbstractStatelessSessionBean implements IHello {

    private static final String beanName = "helloBeanInEJBContext";
    private Hello hello;

    public void setSessionContext(SessionContext context) {
        super.setSessionContext(context);
        setBeanFactoryLocator(ContextSingletonBeanFactoryLocator.
            getInstance("classpath*:beanRefContext.xml"));
        setBeanFactoryLocatorKey("applicationContext-main");
    }

    protected void onEjbCreate() throws CreateException {
        hello = (Hello)getBeanFactory().getBean( beanName );
    }

    public String doSomeBusiness() {
        return hello.doSomeBusiness()+"(Stateless Session Bean)";
    }
}
```



After having edited *HelloBean.java* NetWeaver Developer Studio removes the EJB definition of *HelloBean* from *ejb-jar.xml*!

5. Add *HelloBean* back to *ejb-jar.xml* by selecting it from *J2EE Explorer's EJB Candidates* node. Open the context menu and select "Add to *ejb-jar.xml*".
6. To configure Spring's *ApplicationContext* to use the information about the business object's implementation class described in *applicationContext-ejb.xml* add the following `<env-entry>` to the EJB definition in *ejb-jar.xml*:

ejb-jar.xml

```
<ejb-jar>
  <description>EJB JAR description</description>
  <display-name>EJB JAR</display-name>
  <enterprise-beans>
    <session>
      <ejb-name>HelloBean</ejb-name>
      <local-home>com.sap.spring.ejb.HelloLocalHome</local-home>
      <local>com.sap.spring.ejb.HelloLocal</local>
      <ejb-class>com.sap.spring.ejb.HelloBean</ejb-class>
      <session-type>Stateless</session-type>
      <transaction-type>Container</transaction-type>
    </session>
    <env-entry>
      <env-entry-name>ejb/BeanFactoryPath</env-entry-name>
      <env-entry-type>java.lang.String</env-entry-type>
      <env-entry-value>applicationContext-ejb.xml</env-entry-value>
    </env-entry>
  </enterprise-beans>
</ejb-jar>
```

7. Define JNDI name "HelloBeanJNDI" for *HelloBean* in *ejb-j2ee-engine.xml*.

ejb-j2ee-engine.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE ejb-j2ee-engine SYSTEM "ejb-j2ee-engine.dtd">
<ejb-j2ee-engine>
  <enterprise-beans>
    <enterprise-bean>
      <ejb-name>HelloBean</ejb-name>
      <jndi-name>HelloBeanJNDI</jndi-name>
      <session-props/>
    </enterprise-bean>
  </enterprise-beans>
</ejb-j2ee-engine>
```

Result

Now everything that is needed to deploy the functionality of *Hello.java* as a Stateless Session Bean has been implemented into the EJB Module Project.

Configuring a shared application context and running the application

In the following steps we'll edit the web application's spring configuration so that it can access *helloBean* in the bean definitions in *beanRefContext.xml* and *applicationContext-ejb.xml*.

Furthermore we will setup another application context for the web application only to test if our bean has been deployed as an EJB successfully.

Procedure

1. Remove the bean definition of *helloBean* in *springapp-servlet.xml* since we want to share the bean's definition in *applicationContext-ejb.xml*.
2. In order to test if *helloBean* is also available via a JNDI lookup, create another context definition file *applicationContext-web.xml* in the *WEB-INF* directory and define the bean as follows:
3. In *springapp-servlet.xml* change the *ref bean* parameter of *hello* property in bean id *springappController* to "**helloBeanInEJBContext**" as defined in *applicationContext-ejb.xml*.

applicationContext-web.xml

```
..
<beans>
  <bean id="helloBeanInJNDIContext"
    class="org.springframework.ejb.access.LocalStatelessSessionProxyFactoryBean">
    <property name="jndiName">
      <value>/localejbs/HelloBeanJNDI</value>
    </property>
    <property name="resourceRef">
      <value>>false</value>
    </property>
    <property name="businessInterface">
      <value>com.sap.spring.ejb.IHello</value>
    </property>
  </bean>
</beans>
```

4. Add new *ref bean* property "**helloBeanInJNDIContext**" to inject the bean received from the JNDI lookup.

springapp-servlet.xml

```
..
<beans>
  <bean id="springappController"
    class="com.sap.spring.web.AppController">
    <property name="helloJNDIContext">
      <ref bean="helloBeanInJNDIContext"/>
    </property>
    <property name="helloEJBContext">
      <ref bean="helloBeanInEJBContext"/>
    </property>
  </bean>
  <!--
  <bean id="helloBean" class="com.sap.spring.bl.Hello"/>
  -->
..
</beans>
```

5. Adjust setter methods for dependency injection and controller logic in *AppController.java*.

AppController.java

```
package com.sap.spring.web;

import ..

public class AppController implements Controller {

    private Hello helloEJBContext;
    private IHello helloJNDIContext;

    public ModelAndView handleRequest(HttpServletRequest request,
        HttpServletResponse response) throws ServletException, IOException {

        Map model = new HashMap();
        model.put("helloEJBContextMessage",
            helloEJBContext.doSomeBusiness());
        model.put("helloJNDIContextMessage",
            helloJNDIContext.doSomeBusiness());

        return new ModelAndView("hello", "model", model);
    }

    public void setHelloBeanInEJBContext(Hello helloEJBContext) {
        this.helloEJBContext = helloEJBContext;
    }

    public void setHelloBeanInJNDIContext(IHello helloJNDIContext) {
        this.helloJNDIContext = helloJNDIContext;
    }
}
```

6. Edit hello.jsp so it can display the extended model data.

Hello.jsp

```
<%@ taglib prefix="c" uri="http://java.sun.com/jstl/core" %>
<%@ taglib prefix="fmt" uri="http://java.sun.com/jstl/fmt" %>

<html>
<head><title><fmt:message key="title"/></title></head>
<body>
<h3><fmt:message key="heading"/></h3>

Message from HelloBean in shared EJB context:<br>
    <b><c:out value="${model.helloEJBContextMessage}"/></b>
    <br>
Message from HelloBean in JNDI context:<br>
    <b><c:out value="${model.helloJNDIContextMessage}"/></b>

</body>
</html>
```

7. Finally add context definitions to web.xml. By setting context-parameters *locatorFactorySelector* and *parentContextKey* to the context defined in our EJB Module (*applicationContext-ejb.xml*) becomes the web applications parent context and therefore all beans defined in it are available in the web application.

```

web.xml
..
<web-app>
  <context-param>
    <param-name>locatorFactorySelector</param-name>
    <param-value>classpath*:beanRefContext.xml</param-value>
  </context-param>
  <context-param>
    <param-name>parentContextKey</param-name>
    <param-value>applicationContext-main</param-value>
  </context-param>
  <context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/applicationContext-web.xml</param-value>
  </context-param>
  <servlet>
    <servlet-name>ContextLoaderServlet</servlet-name>
    <servlet-
class>org.springframework.web.context.ContextLoaderServlet</servlet-class>
  </servlet>
  <servlet>
    <servlet-name>springapp</servlet-name>
    <servlet-class>
org.springframework.web.servlet.DispatcherServlet
  </servlet-class>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>springapp</servlet-name>
    <url-pattern>*.htm</url-pattern>
  </servlet-mapping>
  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
  </welcome-file-list>
  <taglib>
    <taglib-uri>/spring</taglib-uri>
    <taglib-location>/WEB-INF/spring.tld</taglib-location>
  </taglib>
</web-app>

```

Result

The web application now is able to access spring beans via application contexts which normally would be out of scope. Furthermore the application exposes its business logic as a local Stateless Session Bean and (for testing reasons only) accesses it via a JNDI lookup (see Figure 8).

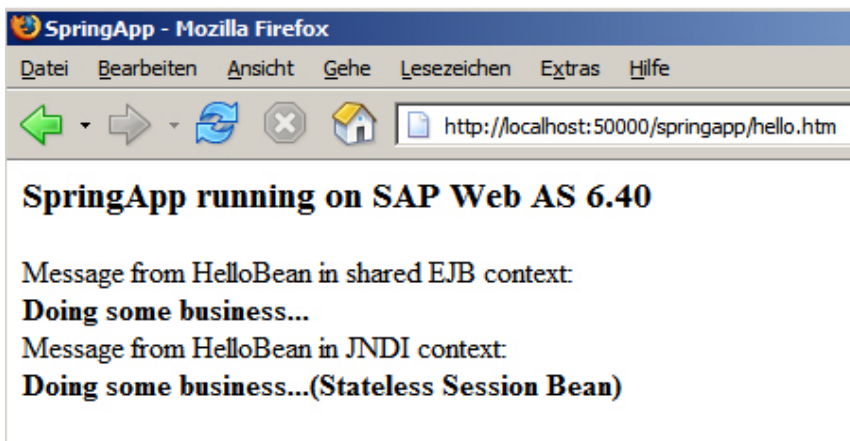


Figure 8 – Spring application using shared application context

About the Author

Helge Martin is an expert in J2EE development and application migration on SAP Web AS 6.40. He has been recently working in the SAP Platform Ecosystem Market Development Engineering Team.